Permissions & RBAC API

Doova One View · Authentication Module

This document explains how the authentication system changed from a simple role field to a full Role-Based Access Control (RBAC) system, and how the frontend must use the permissions list returned by the API to control access to features and routes.

How It Works

The system is built on three layers:

1. **Permission** — an atomic capability codename (e.g. `view_sales`, `export_reports`).
2. **Role** — a named group of permissions (e.g. *manager*, *accountant*). A role bundles together all the capabilities that user type needs.
3. **User** — assigned one or more roles, inheriting every permission from every assigned role. Users with `is_superuser = true` bypass all permission checks on the server.

> ***Note:*** *Permissions are* ***never stored inside the JWT token****. The frontend must call* `GET /auth/profile` *after login to get the current permission list. This guarantees that a role change takes effect immediately — no token re-issue is required.*

After Login — Fetch the User's Permissions

The `POST /auth/login` endpoint returns **only tokens** (access + refresh). It does **not** include roles or permissions. After logging in, immediately call `GET /auth/profile`.

GET	`/auth/profile`	*Returns profile, roles, and permissions*

Request header

```
GET /auth/profile
Authorization: Bearer <access_token>
```

Example response

```
{
"id": "b3f4a912-1234-...",
"email": "ali@example.com",
"full_name": "Ali El Hadi",
"is_active": true,
"created_at": "2025-01-15T09:00:00Z",
"last_login_at": "2025-06-15T10:30:00Z",
"roles": ["manager"],
"permissions": [
"export_accounting", "export_inventory", "export_purchases",
"export_reports", "export_sales", "use_agent",
"view_accounting", "view_inventory", "view_kpi",
"view_master_data", "view_purchases", "view_reports",
"view_sales", "view_tenants", "view_users"
]
}
```


Responses

HTTP Code	Meaning	Body
200	Success	Full UserResponse object (see above)
401	Token missing or expired	{"detail": "Authentication credentials were not provided."}

All Available Permissions

Every permission codename and the feature it gates. The `permissions` array in the API response is always sorted alphabetically.

Codename	Module	Description
view_tenants	Tenants	View tenant list
manage_tenants	Tenants	Create, update, and delete tenants
view_users	Users	View user list and details
manage_users	Users	Create and deactivate users
manage_roles	RBAC	Create roles, assign permissions to roles, assign roles to users
view_sales	Sales	View sales orders and summaries
export_sales	Sales	Export sales data to file
view_purchases	Purchases	View purchase orders
export_purchases	Purchases	Export purchase data to file
view_accounting	Accounting	View journal entries and account moves
export_accounting	Accounting	Export accounting data to file
view_inventory	Inventory	View stock and inventory levels

export_inventory	Inventory	Export inventory data to file
view_kpi	KPI	View KPI dashboards
view_reports	Reports	View generated reports
export_reports	Reports	Download and export reports
use_agent	AI Agent	Send queries to the AI agent
view_master_data	Master Data	View partners, products, and companies

Default Roles

Three roles are seeded by default. Custom roles can be created and managed via the admin RBAC API.

Role	Count	Permissions Included
owner	18 (all)	All permissions — full access to every feature
manager	15	view_tenants, view_users, view_sales, export_sales, view_purchases, export_purchases, view_accounting, export_accounting, view_inventory, export_inventory, view_kpi, view_reports, export_reports, use_agent, view_master_data
accountant	9	view_sales, view_purchases, view_accounting, export_accounting, view_inventory, view_kpi, view_reports, use_agent, view_master_data

How the Frontend Must Use Permissions

After calling `GET /auth/profile`, store the `permissions` array in your global app state (Redux store, Pinia store, React context, etc.). Before rendering any feature or navigating to a guarded route, check whether the required codename is present in that array.

Example (JavaScript / TypeScript)

```
// 1. After login, immediately fetch the user profile
const res = await fetch("/auth/profile", {
headers: { Authorization: `Bearer ${accessToken}` },
});
const profile = await res.json();

// 2. Store permissions in a Set for O(1) lookup
const permissions = new Set(profile.permissions);

// 3. Guard UI elements
const canViewSales = permissions.has("view_sales");
const canExportSales = permissions.has("export_sales");
const canUseAgent = permissions.has("use_agent");

// 4. Guard a route (React Router example)
if (!permissions.has("view_accounting")) {
return <Navigate to="/forbidden" />;
}
```

> ***Note: Superuser:*** *When* `is_superuser` *is* `true` *in the profile response, the server bypasses all permission checks. The frontend may optionally skip permission checks for superusers as well, but the server is always the final authority.*

Handling 403 Errors

When a request is rejected because the user lacks a required permission, the server returns **HTTP 403 Forbidden**. The body always names the missing permission codenames:

```
// 403 Forbidden response body
{
"detail": "Missing permissions: view_sales, export_sales"
}
```

HTTP Code	Meaning	What to show the user
401	Not authenticated — token is missing or expired	Redirect to login page and clear all stored tokens
403	Authenticated but lacks the required permission	Show a "You do not have access to this feature" message — do NOT redirect to login

Common Mistakes to Avoid

x Checking roles instead of permissions	Always check a specific codename (e.g. `permissions.has("view_sales")`), not the role name. A role can be updated to add or remove permissions without being renamed.
x Reading permissions from the JWT	Permissions are **not** encoded in the access token. Decoding the JWT will not give you permissions. Always call `GET /auth/profile`.
x Caching permissions indefinitely	An admin may reassign a user's role at any time. Re-fetch `GET /auth/profile` on each app reload or after a known role change to stay in sync.
x Treating 403 as "not logged in"	**403** = authenticated but lacks permission. **401** = not authenticated. Handle them with different UI flows — a 403 must not redirect to the login page.
x Relying only on frontend guards	The server enforces permissions on every request regardless of what the frontend does. Frontend checks are for UX only — never treat them as a security boundary.
x Wrong Authorization header	Always send `Authorization: Bearer <token>` (capital B, one space after "Bearer"). Missing or malformed headers result in a 401 error.